

The Bean Factory dla LION — raport badawczy nad ewolucyjną metaarchitekturą samoorganizującego się kodu

Teza wykonawcza i rekonstrukcja live repository

Executive Thesis

Na podstawie aktualnego `master` repozytorium `DonkeyJJLove/ai_platform` moja teza jest następująca:

The Bean Factory jest wykonalnym wzorcem docelowym dla LION, ale nie powinna powstać jako nowy równoległy framework agentowy ani jako proste uogólnienie `startup_agent`. Najmocniejsza architektura wynika z syntezy czterech już istniejących linii rozwojowych LION:

`startup_agent` daje zasadę *evidence* → *bottleneck* → *najmniejszy eksperyment* → *najmniejszy build* → *outcome*;

`evolutionary_rnd` / `evolutionary_epoch` daje rygor epistemiczny, falsyfikację, promotion, pamięć i `EvolutionDelta`;

enterprise agent/swarm/fleet daje dynamiczną organizację, capability routing i separację ról;

governed change / builder entry / repository PEP daje kluczową granicę: system może tworzyć własną następną implementację, ale nie może sam ustanowić prawa do jej materializacji.

Wzorec **nie jest nowym znaczeniem klasycznego Spring `BeanFactory`**. Spring `BeanFactory` jest kontenerem/rejestrem definicji komponentów, który tworzy i udostępnia ich instancje. Koncepcja proponowana tutaj idzie znacznie dalej: sama wybiera problem, tworzy falsyfikowalną hipotezę, decyduje czy potrzebny jest nowy komponent, projektuje jego specyfikację, tworzy kandydat implementacji, organizuje koalicję innych komponentów do jego weryfikacji i obserwuje efekt. Nazwa jest więc kolizyjna historycznie, ale semantyka jest inna. ¹

Jednocześnie **nie ma podstaw, by twierdzić, że wszystkie elementy Bean Factory są nowymi wynalazkami**. Autonomic computing od dawna bada self-configuration, self-healing, goal-driven self-assembly i systemy zarządzające sobą według wysokopoziomowych celów; IBM Unity pokazywał nawet goal-driven self-assembly wielu autonomicznych elementów. OSGi posiada dynamiczne publish-find-bind, lifecycle i runtime replacement usług; BB1 posiadał dynamiczne konstruowanie i modyfikowanie planów kontroli podczas działania. ²

Oryginalność architektoniczna LION nie leży więc w samym „self-management” albo „dynamic composition”. Najciekawsza i — w świetle przeanalizowanych źródeł — odróżniająca kombinacja brzmi:

generatywna ewolucja organizacji + jawna epistemika + minimalny eksperyment/build + independent falsification

czyli:

$$Complexity(Result) \gg Complexity(Seed)$$

przy jednoczesnym:

$$Authority(Result) \leq ExplicitAuthorityEnvelope$$

To dokładnie współgra z aktualnym `TARGET_ARCHITECTURE.md`, który ustanawia separację probabilistycznej inteligencji od deterministycznej władzy wykonawczej, rozdziela proposal, authorization i effect, wymaga independent observation/reconciliation oraz definiuje pętlę `TARGET → OBSERVE IMPLEMENTATION → COMPUTE GAP → MISSION → BUILD → VERIFY → INTEGRATE → OBSERVE EFFECT → RECONCILE → NEXT GAP`.

W raporcie używam statusów:

Status	Znaczenie
REPRODUCED	potwierdzone wykonaniem CI lub odtworzonym stanem
OBSERVED	bezpośrednio obecne w live code / Git state
DERIVED	logiczny wniosek z kilku obserwowanych elementów
TARGET	proponowana architektura, nie obecna capability
UNKNOWN	brak wystarczającego evidence
NOT_PROVEN	istnieją przesłanki, ale dowód jest niewystarczający
UNSPECIFIED	implementacja/kontrakt nie definiuje własności

Ten rygor jest zgodny zarówno z Twoją specyfikacją Bean Factory, jak i ze słownikiem statusów `SCIENTIFIC_STATUS.md`. [filecite](#) [turn0file0](#)

Live Repository Reconstruction

Stan `master` został sprawdzony ponownie pod koniec researchu 25 sierpnia 2026. Najnowszy commit nadal był:

```
HEAD
a94882bb80349482c287b76e027c93fe1ed6f1fe

TREE
886fbefc79b82b405e0400e53fe95bfc420f4c67

COMMIT TIME
2026-08-25T08:11:54Z

MESSAGE
```

Merge pull request #188 ...
E004 R17: builder entry permit boundary

Git object dla HEAD wskazuje powyższy tree SHA; commit posiada poprawną weryfikację GitHub.

Evidence	Stan live	Wniosek
Git HEAD	a94882bb80349482c287b76e027c93fe1ed6f1fe	OBSERVED
Git tree	886fbefc79b82b405e0400e53fe95bfc420f4c67	OBSERVED
HEAD message	E004 R17: builder entry permit boundary	OBSERVED
Cyber-Lion Core	completed / success	REPRODUCED przez GitHub Actions
Cyber-Lion Merge Authority Admission	success	REPRODUCED
Bandit Scan Python	success	REPRODUCED
CodeQL Python	success	REPRODUCED
CodeQL Actions	success	REPRODUCED
legacy GitHub combined statuses[]	brak wpisów	nie utożsamiać z brakiem CI; aktualne check-runs są sukcesem
Core job	parse schemas → compile → unittest → startup demo	wszystkie kroki success
Test log	Ran 1283 tests ... OK (skipped=1)	REPRODUCED w CI logu job 97732109263
Actions artifacts dla core run	brak opublikowanych artifact ZIP	OBSERVED ; demo trafia do /tmp, nie do Actions artifact
niezależny lokalny clone/run w moim sandboxie	niedostępny z powodu braku outbound DNS	UNKNOWN , nie udaję drugiej reprodukcji

Check-runs dla tego SHA pokazują pięć zakończonych sukcesem kontroli, w tym dwa CodeQL, Bandit, core i Merge Authority Admission. Workflow Cyber-Lion Core rzeczywiście definiuje parsing JSON schemas, compileall, unittest discover dla cyber_lion/tests oraz osobne uruchomienie python -m cyber_lion.startup_agent.demo; drugi job wykonuje dedykowany test merge-admission.

Istotne jest rozróżnienie: **GitHub CI stanowi aktualnie mocniejszy evidence reprodukcji niż moja lokalna sesja**, ponieważ moja sesja nie mogła niezależnie pobrać repozytorium. Nie przypisuję więc lokalnej reprodukcji statusu `REPRODUCED`.

Dokument architektoniczny sam ustanawia dokładnie taką precedencję: reproduced execution > live code/tests/CI > exact Git state > machine evidence > architecture/governance > history > synthesis.

Najważniejsze pliki źródłowe z tej rekonstrukcji:

Obszar	Exact file	Zbadany zakres	Blob SHA / evidence
Startup model	<code>cyber_lion/startup_agent/models.py</code>	1-260	5c737dce...
Startup evolution	<code>startup_agent/engine.py</code>	1-end	aa75a95f...
Orchestrator	<code>startup_agent/orchestrator.py</code>	1-260	ad269666...
Evidence book	<code>startup_agent/market_intelligence.py</code>	1-240	7e32fe05...
Build planner	<code>startup_agent/build_planner.py</code>	1-320	c051e8c9...
Startup authority	<code>startup_agent/authority.py</code>	1-220	52501b39...
Journal	<code>startup_agent/journal.py</code>	1-240	e3dc9d95...
Local candidate runtime	<code>startup_agent/local_build.py</code>	1-260	9b17c535...
Generic R&D contract	<code>contracts/evolutionary_rnd.py</code>	1-520	b24ad995...
Generic R&D engine	<code>enterprise/evolutionary_rnd.py</code>	1-520	00a0d36b...
Epoch contract	<code>contracts/evolutionary_epoch.py</code>	1-360	78423537...
Epoch engine	<code>enterprise/evolutionary_epoch.py</code>	1-500	live HEAD
Change proposal	<code>contracts/governed_change_proposal.py</code>	1-360	abdb5986...
Builder entry	<code>contracts/builder_entry_permit.py</code>	1-420	0fb43128...
Repository PEP	<code>enterprise/repository_mutation_pep.py</code>	1-520	live HEAD
Agent/Mission/Swarm model	<code>enterprise/models.py</code>	1-280	def82281...
Target architecture	<code>TARGET_ARCHITECTURE.md</code>	1-920	cb969817...
Capability map	<code>CAPABILITY_MAP.md</code>	1-500	862ac31d...

Obszar	Exact file	Zbadany zakres	Blob SHA / evidence
Contract map	CONTRACT_MAP.md	1-520	19feb4d7...
Event model	EVENT_DATA_MODEL.md	1-520	9d9faad8...
Scientific status	SCIENTIFIC_STATUS.md	1-520	55c68fb2...
Repository inventory	REPOSITORY_INVENTORY.md	1-520	live HEAD

Najważniejszy wniosek z samego `master`: **LION jest znacznie dalej niż wskazywałaby analiza samego `startup_agent`**. `startup_agent` jest dobrym embrionem zachowania ewolucyjnego, ale generyczne kontrakty evidence/hypothesis/falsification/promotion/memory oraz ścieżka controlled self-implementation już istnieją poza nim.

Embrion ewolucyjny i formalna definicja wzorca

Existing Evolutionary Embryo

Rzeczywisty przepływ `startup_agent` jest następujący:



`MarketObservation` posiada `source`, `source class`, `observed_at`, `captured_at`, `direction`, `magnitude`, `confidence`, `claim` i `evidence reference`. `MarketEvidenceBook` jest logicznie append-only, deduplikuje zarówno po ID, jak i fingerprint i zachowuje jawne sprzeczności między observations.

`StartupEvolutionAgent` odrzuca stare sygnały, agreguje fresh evidence, nie wymyśla brakujących wymiarów, ocenia hipotezy, wykrywa najniższe wymiary, określa etap i wybiera eksperyment. Aktualna implementacja *startupowa* nadal używa `weighted_score`, czyli jednego agregatu do rankingu hipotez; tego elementu **nie należy przenosić jako uniwersalnego prawa Bean Factory**.

`Experiment` zawiera jawny expected information gain, time-to-evidence, cost, authority class, success metric i stop condition; `velocity_value` odnosi information gain do czasu i kosztu.

Najbardziej wartościowy element `SoftwareBuildPlanner` brzmi dosłownie semantycznie: tworzy *smallest build that can answer the selected experiment question*. Spec wymusza komponenty, interfejsy, acceptance tests, security invariants, non-goals i authority class, a `SafeTemplateBuilder` generuje jedynie mapę plików w pamięci, bez dotykania filesystemu.

`BoundedLocalBuildRunner` dopuszcza wyłącznie `analysis` i `local_prototype`, używa katalogu tymczasowego, waliduje ścieżki, minimalizuje środowisko procesu, nie używa shell i ma timeout; sam moduł jawnie zaznacza jednak, że **nie jest OS security sandboxem**.

Testy potwierdzają między innymi, że `plan()` zwraca stan, eksperyment, build i authority decision; negatywny outcome obniża tylko relewantne wymiary, sukces nie promuje automatycznie wszystkich wymiarów, a wysokiej jakości negatywny wynik może zwiększać learning velocity.

Kluczowe jest jednak to, że **generyczny LION R&D jest obecnie semantycznie dojrzalszy od startup loop**. Kontrakty obejmują już `EvidenceObservation`, `Hypothesis`, `ExperimentProposal`, `SimulationPlan`, `ExperimentResult`, `FalsificationResult`, `EvolutionDelta`, `PromotionDecision` i `RnDMemoryRecord`. `EvolutionDelta` ma wymuszone `authority_effect="NONE"` i `execution_effect="NONE"`, a promotion wiedzy wymaga wspieranej hipotezy, wspierającej falsyfikacji, kompletności contrary evidence, braku unresolved contradictions i canonical PDP evidence.

`EvolutionaryRnDEngine` dodatkowo blokuje stable-ID substitution, replay wyników, niepełny zestaw falsifiers, ukrywanie contrary evidence, rewizję pamięci bez poprawnego previous head i bezpośrednią powierzchnię cognition→effect.

`EvolutionaryEpoch` formalizuje dalszą trajektorię:

```
EPOCH_OPEN
→ OBSERVING
→ HYPOTHESIS_SPACE_ACTIVE
→ TESTING
→ FALSIFICATION_COMPLETE
→ KNOWLEDGE_PROMOTION_READY
→ KNOWLEDGE_PROMOTED
→ MEMORY_COMMITTED
→ DELTA_SYNTHESIZED
→ NEXT_EPOCH_CANDIDATE_READY
```

i również zakazuje niesienia authority/effect przez sam transition.

Mapowanie Startup Evolution Agent → Bean Factory

Live <code>startup_agent</code>	Bean Factory primitive	Disposition	Uzasadnienie
<code>MarketObservation</code>	<code>EvidenceObservation</code> / domain evidence adapter	GENERALIZE	Generic R&D ma silniejszy digest/ provenance model.
<code>MarketEvidenceBook</code>	<code>EvidenceStore</code> / <code>WorldEvidenceView</code>	EXTEND	Zachować dedup, contradiction visibility i freshness.
<code>MarketSignal</code>	derived evidence projection	SUPERSEDE jako core	signed/typed evidence jest lepsze niż kodowanie silnej kontradycji jako niska <code>strength</code> .
<code>ProductHypothesis</code>	<code>Hypothesis</code>	GENERALIZE	Existing generic Hypothesis ma counter-evidence, falsifiers i revisions.
<code>VentureVector</code>	domain-specific <code>StateProjection</code>	KEEP AS ADAPTER	Wagi startupowe nie są prawem Factory.
<code>weakest_dimensions()</code>	<code>GapDetector</code>	GENERALIZE	Gap powinien dotyczyć WORLD/ SYSTEM/GOAL.
<code>weighted_score()</code>	brak uniwersalnego odpowiednika	DO NOT GENERALIZE	Target wymaga multi-criteria/ Pareto.

Live <code>startup_agent</code>	Bean Factory primitive	Disposition	Uzasadnienie
<code>Experiment</code>	<code>ExperimentProposal</code>	GENERALIZE	Generic contract już posiada expected observables i falsification conditions.
<code>choose_experiment()</code>	<code>ExperimentSelector</code>	GENERALIZE	Wybrać information bottleneck, nie etap startupu.
<code>SoftwareBuildSpec</code>	<code>BeanBuildRequest</code> / część <code>BeanSpec</code>	GENERALIZE	Wzorzec minimum-software jest bardzo wartościowy.
<code>SafeTemplateBuilder</code>	<code>BeanCandidateBuilder</code>	EXTEND	Candidate, nie authority.
<code>StartupAuthorityGate</code>	canonical PDP / authority adapters	SUPERSEDE dla consequential paths	Gate startupowy jest prosty; LION ma znacznie silniejszy authority plane.
<code>BoundedLocalBuildRunner</code>	candidate execution provider	KEEP + WRAP	Przydatny lokalnie; jawnie nie jest OS sandboxem.
<code>BuildReceipt</code>	candidate-build evidence	EXTEND	Powinien wiązać spec/ candidate/ runtime digests.
<code>ExperimentOutcome</code>	<code>ExperimentResult</code>	GENERALIZE	Generic R&D ma result classes i observer evidence.

Live <code>startup_agent</code>	Bean Factory primitive	Disposition	Uzasadnienie
<code>apply_outcome()</code>	state reconciliation / <code>EvolutionDelta</code> synthesis	GENERALIZE	Korekta musi być ograniczona do evidence.
<code>EvolutionJournal</code>	episodic evolution history	SUPERSEDE/ADAPT	Generic R&D ma hash-chained memory z negatywnym evidence.
provider coordinator	Capability Provider Fabric	GENERALIZE	Provider output $\neq$ authority.
<code>CyclePlan</code>	<code>EpochWorkPlan</code>	GENERALIZE	Jedna domena $\rightarrow$ domain-independent epoch.

Formal Definition

Proponowana precyzyjna definicja:

The Bean Factory jest governowaną metaarchitekturą ewolucyjną, która z jawnego celu, obserwowanego świata i obserwowanego stanu systemu wyprowadza falsyfikowalne różnice, tworzy najmniejsze specyfikacje potrzebnych capabilities, materializuje je jako nieuprzywilejowanych kandydatów, dynamicznie komponuje tymczasowe organizacje do ich budowy/weryfikacji/obserwacji, a następnie — wyłącznie przez zewnętrzny authority plane — dopuszcza ograniczone skutki, obserwuje ich rzeczywisty rezultat i aktualizuje pamięć oraz kolejną epokę.

Formalnie:

$$W_t = \text{WorldState}, \quad S_t = \text{SystemState}, \quad G_t = \text{GoalState}$$

$$D_t = \Delta(W_t, S_t, G_t, E_t)$$

$$H_t = \text{Hypothesize}(D_t, E_t)$$

$$X_t = \text{SelectExperiment}(H_t, E_t, U_t)$$

$$B_t = \text{MinimalBeanNeed}(X_t)$$

$$O_t = \text{Compose}(B_t, \text{Capabilities}_t, \text{Constraints}_t)$$

$$C_t = \text{BuildCandidate}(B_t, O_t)$$

lecz:

$$C_t \not\approx \text{Authority}(C_t)$$

Dopiero:

$$Grant_t = PDP(Identity, Mission, Currentness, Policy, Observability, Risk, CandidateEvidence)$$

i:

$$Effect_t = PEP(C_t, Grant_t)$$

po czym:

$$R_t = \text{Reconcile}(ExpectedEffect_t, ObservedEffect_t)$$

$$S_{t+1}, M_{t+1}, D_{t+1} = \text{Evolve}(R_t, E_t)$$

To jest **self-implementation without self-authorization**. Ta separacja nie jest jedynie projektem: current LION już wymusza, że `EvolutionDelta`, epoch transition i `GovernedChangeProposal` nie niosą effect authority.

Seed, Bean i architektura Factory

Seed Architecture

Minimalny Seed nie powinien zawierać gotowego produktu, gotowego workflow ani katalogu organizacji. Powinien zawierać **gramatykę bezpiecznej ewolucji**.

Proponuję następujące minimum:

Seed primitive	Exact responsibility	Live reusable base	Status
<code>EvidenceObservation</code>	canonical observation + provenance + digest	<code>contracts/evolutionary_rnd.py</code>	OBSERVED/ REUSE
<code>GoalContract</code>	jawny cel, constraints, success/stop/defer	częściowo <code>MissionSpec</code> ; brak uniwersalnego root-goal contract	TARGET
<code>WorldSnapshot</code>	current external state projection	startup market evidence jako jeden adapter	TARGET
<code>SystemSnapshot</code>	current live system projection	architektura opisuje potrzebę; brak jednego canonical contract znalezonego w badaniu	TARGET / UNKNOWN implementation
<code>Gap</code>	różnica W/S/G	<code>EvolutionDelta</code> jest późniejszą deltą, nie pełnym gap contract	TARGET

Seed primitive	Exact responsibility	Live reusable base	Status
Hypothesis + Falsification	teoria zmiany i próba jej obalenia	istnieje	OBSERVED/ REUSE
EvolutionEpoch	deterministyczna state machine	istnieje	OBSERVED/ REUSE
BeanSpec	canonical capability unit	brak canonical BeanSpec w code search; istnieje węższy AgentSpec	TARGET
CompositionContract	jawna relacja wielu Beanów	istnieją domain-specific fleet compositions, nie generic Bean composition	TARGET
CapabilityResolver	discover/filter capabilities	capability architecture istnieje, częściowe implementacje enterprise	EXTEND
Authority boundary	PDP + typed grant + PEP	silnie rozwinięte	REUSE
Observer/Reconciler	independent effect evidence	istnieją konkretne repository/runtime flows	REUSE/ GENERALIZE
EvolutionMemory	episodic + promoted + negative evidence	RnDMemoryRecord	REUSE/EXTEND
Supersession	lineage replacement without history deletion	istnieją mechanizmy hypotheses/memory	REUSE/ GENERALIZE

Ważna redukcja: **LLM nie należy do Seed**. Model jest providerem inference. Seed powinien móc walidować kontrakty, authority, lineage i transition nawet wtedy, gdy provider modelowy zostanie zastąpiony.

To jest spójne z `CAPABILITY_MAP.md`: capability jest zdolnością rozwiązania klasy problemu, a nie repozytorium, usługą lub zapamiętanym workflow; discovery nie oznacza permission.

Bean Model

Najważniejsze rozdzielenie:

```

BeanSpec
  !=
BeanCandidate
  !=
BeanInstance
  !=

```

```

BeanRuntimeIdentity
    !=
BeanAuthorityEnvelope

```

`AgentSpec` już dziś rozdziela definicję organizacyjną agenta od wykonania i zawiera capabilities, authority ceiling, observability, memory policy, runtime/cost budget i risk class. `MissionSpec` opisuje input dynamicznej organizacji, a `SwarmSpec` jest derived time-bounded organizational mosaic. To jest bardzo dobry prekursor modelu Bean, ale Agent pozostaje tylko jednym typem Beana.

Proponowany lifecycle:

```

DRAFT
  ↓ seal
SEALED_SPEC
  ↓ generate
CANDIDATE
  ↓ verify
VERIFIED_CANDIDATE
  ↓ admission
ADMITTED
  ↓ runtime bind
RUNNING
  ↓
QUIESCING
  ├── SUPERSEDED
  ├── REVOKED
  └── FAILED

```

`CANDIDATE → VERIFIED` **nie daje authority.**

`VERIFIED → ADMITTED` **nie dowodzi efektu.**

`RUNNING → SUCCESS` **nie daje closure.**

Closure pojawia się dopiero po observation + reconciliation, zgodnie z targetem LION.

BeanSpec schema

Grupa	Pole	Znaczenie / constraint
identity	<code>bean_id</code>	stabilna logiczna tożsamość
identity	<code>bean_type</code>	np. <code>agent</code> , <code>observer</code> , <code>builder</code> , <code>verifier</code> , <code>adapter</code> , <code>deterministic_service</code> , <code>workflow</code>
identity	<code>version</code>	immutable spec version
lineage	<code>lineage</code>	parent spec digests + derivation kind
purpose	<code>purpose</code>	funkcja bez implicit authority
goal	<code>goal_contract</code>	goal/ref + immutable goal digest

Grupa	Pole	Znaczenie / constraint
goal	success_condition	observable, testable
goal	stop_condition	jawne zakończenie
goal	defer_condition	kiedy legalny jest brak decyzji
interface	inputs	typed input schemas
interface	outputs	typed output schemas
interface	interfaces	declared ports/protocols
capability	required_capabilities	zależności logiczne
capability	provided_capabilities	deklarowane zdolności
authority	authority_ceiling	ceiling , nigdy grant
authority	required_grants	klasy wymaganych zewnętrznych grantów
epistemic	epistemic_requirements	dopuszczalne epistemic classes
epistemic	evidence_requirements	minimum evidence
provenance	provenance_policy	wymagania upstream refs/digests
memory	memory_policy	read/write/promotion classes
context	context_policy	trust/type isolation
observation	observability_requirements	required observer channels/quorum
resource	resource_budget	CPU/RAM/IO/etc.
resource	cost_budget	monetary/logical budget
resource	time_budget	deadline/runtime
runtime	runtime_class	provider/runtime requirements
runtime	sandbox_class	required isolation
graph	dependencies	exact Bean/capability deps
graph	compatibility_constraints	version/interface constraints
failure	failure_modes	declared anticipated failures
failure	degradation_policy	explicit degraded behavior
failure	revocation_policy	kill/freeze/revoke semantics
security	security_invariants	invariants enforced/tested
validation	acceptance_tests	positive contract
validation	falsification_conditions	what disproves usefulness/correctness

Grupa	Pole	Znaczenie / constraint
evolution	<code>evolution_hooks</code>	observable triggers, never arbitrary code authority
evolution	<code>replacement_policy</code>	migration rule
evolution	<code>supersession_policy</code>	lineage/reconciliation prerequisites
integrity	<code>implementation_digest</code>	optional pinned impl, required before candidate admission
integrity	<code>spec_digest</code>	domain-separated canonical digest

Minimalny constraint:

spec_digest = SHA256("LION/BEAN – SPEC/1\0" || canonical_json(spec \ spec_digest))

Przykład: Bean obserwacyjny

```

bean_id: market-observer
bean_type: observer
version: 1.0.0
purpose: Collect fresh external market evidence
provided_capabilities:
  - evidence.market.observe
authority_ceiling: read
required_grants: []
success_condition: "emits sealed EvidenceObservation"
stop_condition: "evidence budget exhausted"
defer_condition: "source currentness cannot be established"
runtime_class: provider
sandbox_class: network-readonly
observability_requirements:
  - source_ref
  - observed_at
  - captured_at
  - content_digest
security_invariants:
  - "observation cannot mint authority"
falsification_conditions:
  - "source provenance cannot be reconstructed"

```

To jest naturalna generalizacja `MarketObservation`, lecz powinno użyć canonical digest/provenance z generic R&D.

Przykład: Builder Bean

```

bean_id: detached-candidate-builder
bean_type: builder
version: 1.0.0

```

```

purpose: Materialize a detached candidate for an admitted build request
provided_capabilities:
  - candidate.build.detached
authority_ceiling: local_write
required_grants:
  - builder_entry_permit
runtime_class: trusted-builder
sandbox_class: detached-worktree
security_invariants:
  - "repository_ref_effect == NONE"
  - "external_effect == NONE"
  - "candidate_scope cannot expand"
  - "builder cannot self-verify consequential attachment"

```

Ten przykład odpowiada już istniejącej filozofii `BuilderEntryPermit`: permit wiąże repository, baseline SHA/tree, scope, current authority state, trusted builder identity/implementation/attestation, a jednocześnie wymusza `authority_effect`, `execution_effect`, `repository_ref_effect` i `external_effect` na `NONE`.

Lineage

Rozróżniałbym dokładnie:

```

INstantiate
same BeanSpec → new BeanInstance

Clone
new instance state derived from another instance
(no new spec semantics)

Derive
new BeanSpec from parent BeanSpec

Fork
new independent lineage from prior spec

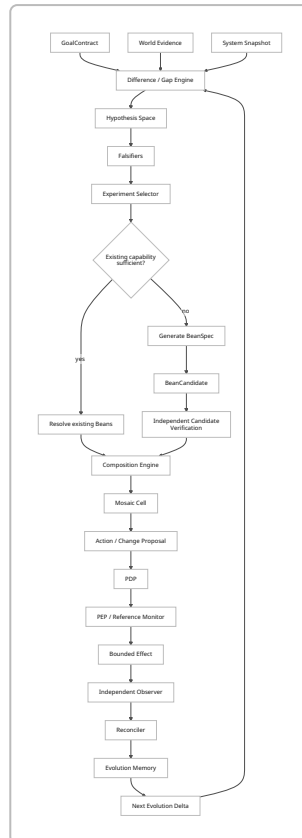
Generate
new BeanSpec without a direct same-role parent

Supersede
explicit replacement relation after compatibility + evidence decision

```

Potomek nie może zmienić historycznego rodzica. Istniejący R&D model już wymaga, by revision > 1 wskazywała exact superseded hypothesis digest; memory jest również hash-chainowana.

Factory Architecture



Najważniejsza granica Factory:

Factory może wytworzyć opis, kod, test, organizację i proposal. Nie może wytworzyć obowiązującego authority dla tego proposal.

CompositionContract

Proponowany kontrakt:

```

CompositionContract
  composition_id
  mission_id
  goal_digest

  bean_bindings[]
  capability_requirements[]
  interface_bindings[]

  relations[]
    REQUIRES
    PROVIDES
    CONFLICTS_WITH
    DEPENDS_ON
    VALIDATES
    OBSERVES
    RECONCILES
  
```

SUPERSEDES

```

authority_constraints[]
runtime_constraints[]
resource_budget
observability_budget

independence_requirements[]
currentness_requirements[]

compatibility_evidence_refs[]
provenance_refs[]

composition_digest

```

Nie należy mylić go z istniejącym `RuntimeCompositionConfig` dla F005-H: ten obiekt wiąże konkretne runtime stores i trust pins oraz jawnie nie tworzy mission facts, results, heartbeats, reconciliation facts, authority, leases, effects ani closure evidence. Jest dobrym przykładem **composition-root binding**, ale nie jest ogólną semantyką Bean composition.

CompositionEngine rules

Reguła	Formalizacja	Przykład deny
Capability closure	$Required(C) \subseteq Provided(C) \cup ExternalAllowed$	brak verifier capability
Interface compatibility	output schema A $\models$ input schema B	producer <code>v2</code> niezgodny z consumer <code>v1</code>
Dependency graph	domyślnie DAG	$A \rightarrow B \rightarrow C \rightarrow A$
Conflict exclusion	żadna aktywna para <code>CONFLICTS_WITH</code>	dwa exclusive writers
Mission binding	wszystkie Beans przypięte do tej samej misji lub jawnej submisji	przypadkowy provider z innej misji
Authority non-amplification	każdy efekt $\leq$ mission grant i performer ceiling	composition „sumuje” credentials dwóch agentów
No credential inheritance	membership nie tworzy grant	verifier dziedziczy token buildera
Builder/verifier separation	dla consequential effect builder $\neq$ sole final verifier	self-signing build
Observer independence	effect producer $\neq$ jedyny observer	executor sam deklaruje ref result
Currentness	wymagane bindings current near effect time	stary grant po revoke
Budget closure	suma/rezerwacja $\leq$ mission budget	dynamiczne N workerów przekracza quota

Reguła	Formalizacja	Przykład deny
Observability	required budget $\leq$ available observer state	brak ref observera przed Git attach
Supersession lineage	replacement wiąże exact prior digest	podmiana pod tym samym ID
Reconfiguration cost	reorganizacja tylko gdy uzasadniona evidence	swarm thrash co cykl

Dla authority najbezpieczniejsza formalizacja nie jest prostym „intersection całego swarmu”. Różni członkowie mają różne role. Zamiast tego:

$\forall effect\ e\ executed\ by\ b :$

$$A_{effective}(b, e) \subseteq A_{mission}(e) \cap A_{ceiling}(b, e) \cap A_{externalGrant}(b, e) \cap A_{current}(e) \cap A_{observable}(e)$$

Sama Composition:

$$A_{minted}(Composition) = \emptyset$$

Aktualny target już ustanawia `child <= parent`, `executor <= mission`, `mission <= fleet envelope` i wyraźnie stwierdza, że swarm membership nie oznacza credential inheritance.

Ewolucja, Mosaik i protokół samoimplementacji

Evolution Engine

Nie przenosiłbym startupowego `weighted_score()` do Bean Factory. `VentureVector` może nadal być bardzo dobrym **domain projection**, ale engine ogólny powinien operować na strukturze kryteriów.

Proponuję:

$$P_t = \langle G, C, U, I, R, A, O, V, D, K \rangle$$

gdzie:

- G = gap severity,
- C = evidence currentness/completeness,
- U = uncertainty,
- I = expected information gain,
- R = risk,
- A = authority exposure/cost,
- O = observability deficit,
- V = reversibility,
- D = dependency urgency,
- K = reconfiguration/resource cost.

To **wektor**, nie **score**.

Decyzja ma trzy warstwy:

HARD ADMISSION FILTERS

security invariants
authority feasibility
currentness
required observability
budget
falsifiability

↓

PARETO FRONTIER

information gain ↑
gap closure ↑
risk ↓
authority exposure ↓
cost ↓
reconfiguration cost ↓

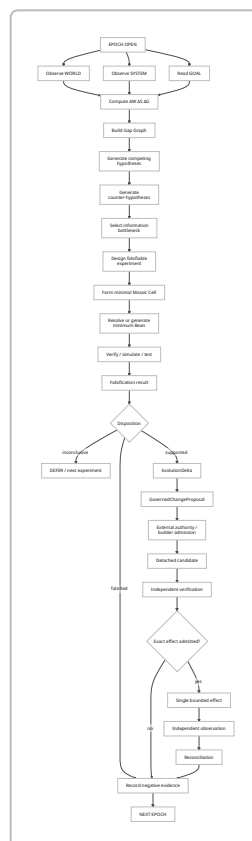
↓

DETERMINISTIC TIE BREAK

stable IDs / digests / explicit policy priority

Dzięki temu eksperyment o minimalnie lepszej wartości biznesowej nie wygra automatycznie z wariantem o dramatycznie mniejszym authority exposure.

Epoch loop



Legalne outcome'y:

```

NO_CHANGE
DEFER
FALSIFIED
INCONCLUSIVE
REJECTED
BUILD_CANDIDATE
SUPERSEDE
NEXT_EXPERIMENT
NEXT_EPOCH_CANDIDATE_READY

```

Istniejący epoch engine już ma explicite `BLOCKED` i `UNKNOWN`, a `assert_next_epoch_ready()` wymaga poprawnego delta lineage, verified promotion PDP i committed memory before next epoch readiness.

Observability budget

Nie reprezentowałbym observability jednym `0..1`. Dla każdego consequential effect:

```

observability_budget:
  required_observers: 2
  independent_from_executor: true
  required_channels:
    - effect_target_state
    - runtime_identity
    - authority_decision
  max_staleness_seconds: 5
  attribution_required: true
  pre_effect_observation: true
  post_effect_observation: true
  reconciliation_required: true
  unknown_policy: DENY

```

Warunek admission:

$$ObsAvailable(e, t) \succeq ObsRequired(e)$$

Jeśli nie:

$$Impact(e) \uparrow \wedge ObsAvailable \downarrow \Rightarrow Authority(e) \downarrow$$

Docelowa architektura już ustanawia zasadę `LOSS OF REQUIRED OBSERVATION → NO NEW HIGH-IMPACT AUTHORITY`.

Mosaic Organization

Live enterprise model posiada już znaczącą część docelowej semantyki: `MissionSpec` zawiera required capabilities, authority ceiling, risk, max agents, observability quorum i opcjonalny independent verifier; `SwarmSpec` jest „derived, time-bounded organizational mosaic for one mission”, a RED swarm wymaga verifera.

Bean Factory powinna tę zasadę rozszerzyć z „agentów” do heterogenicznych Beans:

```

PROBLEM
↓
required capability set
↓
existing capability graph
↓
candidate implementations
↓
compatibility + authority + independence filters
↓
minimal non-dominated composition
↓
MOSAIC CELL

```

Przykład:

Rola logiczna	Bean	Potrzebny dlatego, że
Explorer	model/inference Bean	generuje competing hypotheses
Evidence source	observer/provider Bean	wiąże teorię z current world
Builder	deterministic/tool Bean	materializuje candidate
Verifier	independent verifier Bean	próbuję złamać candidate
Observer	runtime/repository observer	mierzy realny effect
Reconciler	deterministic Bean	expected vs observed
Authority source	nie Bean tworzony przez Factory	konstytucyjna granica systemu

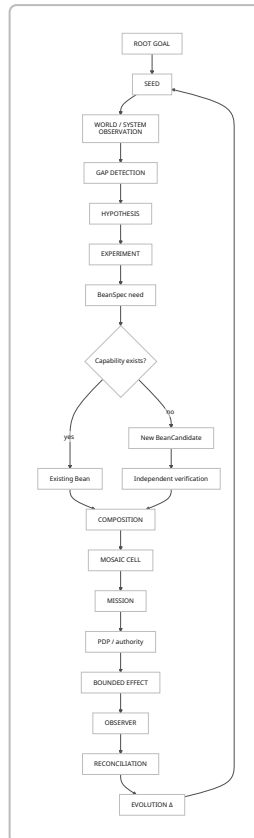
Organizacyjna funkcja może być modelowana:

$$O_{t+1} = \Phi(O_t, \Delta W_t, \Delta S_t, \Delta G_t, E_t, A_t, Obs_t)$$

ale pod hard constraints:

$$SecurityInvariants(O_{t+1}) = TRUE$$

Growth graph



Self-Implementation Protocol

Na tym polu live LION jest już szczególnie silny.

`EvolutionDelta` nie może przenosić authority.

`GovernedChangeProposal` jest canonical non-effectful engineering intent, wiąże exact delta/epoch/memory/promotion/PDP evidence i również wymusza `authority_effect="NONE"` oraz `execution_effect="NONE"`.

Najnowszy `BuilderEntryPermit` wiąże baseline master SHA i tree, scope, authority epoch/state, root grant, trusted builder identity/implementation/attestation i nadal nie pozwala na repository-ref effect.

Repository Mutation PEP idzie jeszcze dalej: composition-root jest właścicielem verification source, exact-CAS effect portu, independent observera, trusted clock, runtime scope i local journal; authority jest ponownie walidowane przy effect time. Sam moduł ostrożnie zaznacza, że jest reference enforcement core i **nie dowodzi globalnego complete mediation/L2**.

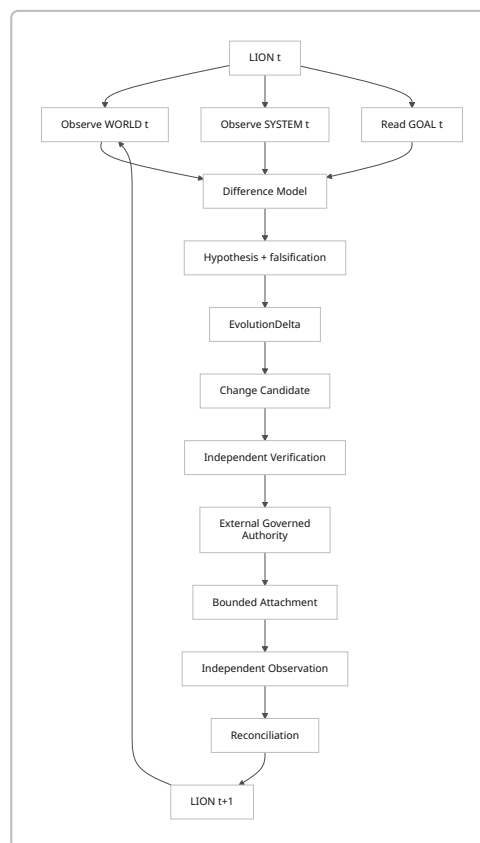
Docelowy protokół Bean Factory powinien więc **reuse**, a nie zastępować tę trajektorię:

```

OBSERVED GAP
→ HYPOTHESIS
→ EXPERIMENT
→ FALSIFICATION
  
```

- EVOLUTION DELTA
- GOVERNED CHANGE PROPOSAL
- BUILDER ENTRY PERMIT
- DETACHED CANDIDATE
- INDEPENDENT VERIFICATION
- EFFECT-SPECIFIC PDP/GRANT
- EFFECT-TIME CURRENTNESS
- PEP / REFERENCE MONITOR
- EXACT SINGLE EFFECT
- INDEPENDENT OBSERVATION
- RECONCILIATION
- MEMORY PROMOTION
- SUPERSESSION
- NEXT EPOCH

Self-development graph



Self-update / Self-upgrade / Self-evolution policies

Klasa	Definicja	Promotion requirements	Automation ceiling
self-update	implementation change bez zmiany capability/ architecture contract	exact baseline, tests, independent verification, compatibility, effect-specific permit	wysoka automatyzacja możliwa dla niskiego ryzyka

Klasa	Definicja	Promotion requirements	Automation ceiling
self-upgrade	nowa capability/ interface/provider	wszystko z update + capability tests + composition tests + observer + rollback/ supersession	średnia; nowa consequential capability wymaga jawnego authority
self-evolution	zmiana decomposition, architecture boundary, governance lub evolution operatora	multi-epoch evidence, competing hypotheses, architecture tests, independent verifier, external policy approval	niska dla governance; nie self-authorized
root-goal change	zmiana WHAT THE GOAL IS	external constitutional authority	zero autonomous promotion
reference- monitor change	zmiana enforcement constitution	independent external authority + parallel verifier + regression/falsification	zero self-approval
authority ceiling change	zwiększenie możliwego effect space	external authority source	zero self-minting

Authority, evidence, pamięć i rekonsyliacja

Authority Architecture

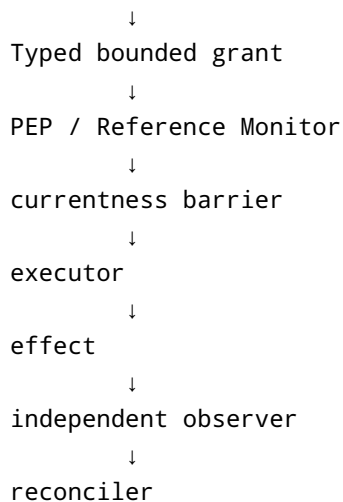
Obecna architektura LION już jednoznacznie mówi:

```
SEM proposal
!=
MAND authorization
!=
INF effect
```

oraz wymaga PEP/reference-monitor boundary dla consequential effects.

Dla Bean Factory proponuję kanoniczną ścieżkę:

```
Bean inference / organization
↓
ActionProposal
↓
Intent normalization
↓
Mission + runtime identity + exact baseline
↓
PDP
```



PDP odpowiada **czy** authority wolno przyznać.

PEP odpowiada **czy ten konkretny effect** mieści się dokładnie w aktualnym grant.

Executor nie odpowiada na żadne z tych pytań.

Observability & Reconciliation

Event model posiada już konieczne intuicje: consequential `ActionExecuted` powinien wskazywać applied `GateApplied`, `MemoryCommitted` wymaga policy/provenance/source candidate, a utrata wymaganej observability przed kolejnym high-authority action powinna wywołać authority degradation.

Execution receipt ma być append-only evidence; correction ma supersedować historyczny receipt zamiast go mutować. Symulacja jest jawnie odróżniona od observed fact.

Proponowana Bean evidence schema:

```

schema_version: 1.0.0
evidence_id: ...
evidence_kind: observation|execution|verification|falsification|
reconciliation
subject_ref: ...
source:
  entity_id: ...
  source_class: ...
  implementation_digest: ...
observed_at: ...
captured_at: ...
epistemic_class: OBSERVED
content_digest: ...
provenance_refs: [...]
correlation_id: ...
causation_id: ...
limitations: [...]
contradictions: [...]
currentness:

```

```

    valid_at: ...
    expires_at: ...
  observer:
    identity_digest: ...
    independence_class: ...
  evidence_digest: ...

```

Epistemic taxonomy

Live generic R&D zawiera już OBSERVED, DERIVED, CALIBRATED, ASSUMED, HYPOTHESIS, SPECULATION, STRESS_PARAMETER, SIMULATED; result classes obejmują m.in. OBSERVED, SIMULATED, REPRODUCED, FAILED, UNKNOWN, PARTIAL_UNKNOWN.

Dla Bean Factory rekomenduję wystawiać wspólny superset:

```

OBSERVED
ANCHORED
DERIVED
CALIBRATED
ASSUMED
HYPOTHESIS
SPECULATION
STRESS_PARAMETER
SIMULATED
REPRODUCED
FAILED
UNKNOWN
PARTIAL_UNKNOWN
CONTRADICTED
SUPERSEDED

```

ANCHORED i CONTRADICTED wymagają jeszcze dokładnej decyzji kompatybilności ze słownikiem live — **TARGET**, nie twierdzą, że obecnie są canonical R&D classes.

Retention rules

Evidence class	Retention
external observations used in decision	co najmniej przez cały lineage wszystkich zależnych decisions
negative result	nieusuwalny przez promotion/summary; może być superseded, nie erased
falsification	retained with exact hypothesis/result digests
authority decision	retained przez okres effect/audit lineage
execution receipt	append-only
reconciliation	append-only, correction via supersession

Evidence class	Retention
stale evidence	zachować historycznie, ale wyłączyć z current inference po currentness policy
raw high-volume telemetry	może być compacted, lecz promoted summary musi zachować upstream content digest/reference
rejected candidate	utrzymać przynajmniej digest, failure class i relevant negative evidence
superseded Bean	utrzymać lineage + replacement digest + reason

To współgra z istniejącym `RnDMemoryRecord`, który ma `negative_evidence_refs`, `supersedes_memory_digest`, previous head i digest chain.

Memory & Supersession

Rozdzielenie powinno być:

EPISODIC EVOLUTION MEMORY

what happened

PROMOTED STRUCTURAL MEMORY

what is admitted to influence future architecture

POLICY / AUTHORITY

what may happen

czyli:

Memory / Policy / ~~Authority~~

To jest już jawna reguła target architecture.

Promoted memory nie może być „summary of successes”. Promotion musi zachować:

supporting evidence
counter evidence
negative experiments
known limitations
falsification results
superseded hypotheses
dependency state
policy decision

Aktualny `PromotionDecision` bardzo dobrze wymusza zasadę „no hidden contrary evidence”: knowledge promotion wymaga zero unresolved contradictions i complete contrary evidence.

Bezpieczeństwo, falsyfikacja i konkurencyjne architektury

Failure Model

Failure	Required behavior	Test
bad Bean generation	candidate rejected, no admission	malformed schema/property test
false capability declaration	composition deny	provider advertises capability but contract probe fails
dependency cycle	deny composition	graph fuzz $A \rightarrow B \rightarrow C \rightarrow A$
dependency explosion	budget/block	generated fan-out stress
Bean monoculture	flag epistemic correlation	multiple „verifiers” same model/impl digest
compromised verifier	require independent evidence where impact class requires	verifier lies vs observer
compromised observer	UNKNOWN/DEGRADED, no high-impact continuation	observer returns stale/forged state
goal drift	stop evolution	mutated goal digest during epoch
stale world model	STALE, do not promote	currentness expiry
stale system model	rebuild snapshot / block effect	HEAD differs from bound tree
contradictory goals	DEFER/BLOCKED	incompatible immutable goal constraints
metric gaming	retain root success contract	Bean tries to rewrite metric after failure
false-positive gap	falsification yields no-change	adversarial irrelevant delta
false-negative gap	multiple observer/source test	hide system defect from one sensor
runaway Bean generation	global generation/resource budget	recursive „need another Bean”
organization thrashing	reconfiguration hysteresis/cost	alternating capability availability
memory poisoning	candidate-only memory + PDP promotion	untrusted text claims policy
policy drift	effect-time currentness deny	policy version changes after planning
provider drift	digest/currentness mismatch	provider swaps implementation
authority laundering	deny transitive credential transfer	A delegates to B then $B \rightarrow C$

Failure	Required behavior	Test
self-verification	deny consequential promotion	builder signs own final verification
test weakening	compare test/security invariant delta	candidate removes failing test
replay	single-use IDs/digest journal	repeat same effect/admission
duplicate Bean instance	lifecycle conflict/quarantine	same runtime identity twice
duplicate mission	canonical mission identity deny	double scheduler registration
lost heartbeat	freeze/reassign without authority increase	kill worker
network partition	degraded/frozen state	stale authority partition
unreconciled effect	mission cannot close	effect succeeded, observer missing
Factory crash mid-evolution	resume from append-only state or block	crash injection
incompatible Factory upgrade	old contract compatibility gate	version N/N+1 corpus tests

Target architecture already wymienia m.in. duplicate executor/mission/lease, stale baseline, lost heartbeat, forged/stale attestation, dependency cycle, partial network partition, observer/verifier death, reconciliation disagreement, telemetry loss, credential leakage, provider drift, policy drift i replay; każde ma `FAIL CLOSED` albo `EXPLICIT DEGRADED STATE`.

Security Invariants & red-team results

BF-I1–BF-I10 są obecnie **TARGET Bean-specific invariants**, ponieważ canonical Bean nie istnieje jeszcze; wiele z nich ma jednak bezpośrednie analogi już egzekwowane w LION.

Invariant	Falsyfikacja/atak	Live analogue	Wymagany wynik
BF-I1 Factory cannot mint authority for a Bean it creates	generated Bean zawiera grant/token	<code>EvolutionDelta</code> /proposal authority NONE	DENY
BF-I2 Bean cannot modify own authority ceiling	candidate zmienia ceiling read→deploy	Agent authority ceiling + external gate model	DENY
BF-I3 Bean cannot redefine success after failure	failed test → candidate edits success metric	nie ma jeszcze Bean contract	DENY; metric digest immutable

Invariant	Falsyfikacja/atak	Live analogue	Wymagany wynik
BF-I4 Composition cannot acquire ungranted capability authority	credential union via swarm	target: swarm membership != credential inheritance	DENY
BF-I5 generated code untrusted until independent verification	builder marks itself verified	builder-entry separated from repo effect	candidate only
BF-I6 Factory evolution cannot disable its own verifier	patch removes verifier requirement	partial architecture analogue	external governance required
BF-I7 root goal requires external authority	model changes goal to easier one	goal constitution not yet canonical	hard DENY
BF-I8 negative evidence cannot disappear during compaction	summary omits falsified result	<code>negative_evidence_refs</code> , contrary completeness	promotion DENY
BF-I9 newer Bean does not supersede without lineage	same ID/payload swap	R&D identity substitution + supersession lineage	DENY
BF-I10 cell failure cannot escalate privileges	verifier crashes → builder gains verifier role	target loss of observation cannot raise authority	DEGRADED/ FROZEN

Dodatkowe wymagane testy:

```

property-based:
  authority_effect always NONE for epistemic objects
  lineage acyclic
  spec digest deterministic
  composition cannot widen scope

mutation:
  flip NONE → write
  remove currentness check
  remove verifier independence
  remove negative evidence requirement
  weaken success metric
  replace exact digest comparison with prefix comparison

fuzz:
  IDs

```

paths
 capability graphs
 versions
 resource scopes
 provenance graphs
 replay identifiers
 malformed timestamps
 Unicode / separator confusion

Najsilniejszym obecnym wzorcem negatywnego testowania jest repository PEP: wiąże exact candidate, exact expected ref, exact mission, exact grant, exact runtime scope i pinned independent dependencies, a błędy admission zamienia na DENY.

Competing Architectures

Wariant	Co wyjaśnia dobrze	Próba falsyfikacji jako pełnego wyjaśnienia Bean Factory	Wynik
Spring BeanFactory / IoC	definicje, lifecycle, dependency injection, creation	nie generuje hypothesis/experiment/falsification/authority architecture	niewystarczający 3
OSGi dynamic services	publish-find-bind, dynamic lifecycle, versioning, secure service access	nie definiuje epistemic evolution ani governance self-implementation	bardzo bliski composition layer, ale niewystarczający 4
Autonomic computing / MAPE-like feedback	monitor/analyze/plan/execute and self-management	klasyczne self-management nie wymusza LION-owego oddzielenia generated change od external authority constitution	bliski Evolution Engine 5
IBM Unity autonomic elements	goal-driven self-assembly, decentralization, self-healing/optimization	organizacja może się samo-assemble, ale to nadal nie daje Bean Factory epistemic/falsification + governed-code lineage	bliski Mosaic 6
Blackboard / BB1	dynamic control planning, shared problem state, learned heuristics	brak obecnego modelu immutable authority/effect/reconciliation	bliski reasoning/control seed 7
Collaborative multi-agent systems	autonomous communicating entities, dynamic cooperation	Agent ≠ general Bean; deterministic component/verifier/policy adapter nie musi być agentem	częściowe 8

Wariant	Co wyjaśnia dobrze	Próba falsyfikacji jako pełnego wyjaśnienia Bean Factory	Wynik
Dynamic reconfiguration	runtime replacement and consistency analysis	zwykle odpowiada „jak rekonfigurować”, nie „dlaczego epistemicznie wygenerować nowy komponent”	częściowe ⁹

Najlepsza falsyfikacja tezy o „nowym wzorcu” brzmi więc:

Gdyby Bean Factory sprowadzała się tylko do dynamicznego discovery, lifecycle i replacement komponentów, byłaby przede wszystkim OSGi/autonomic architecture pod nową nazwą.

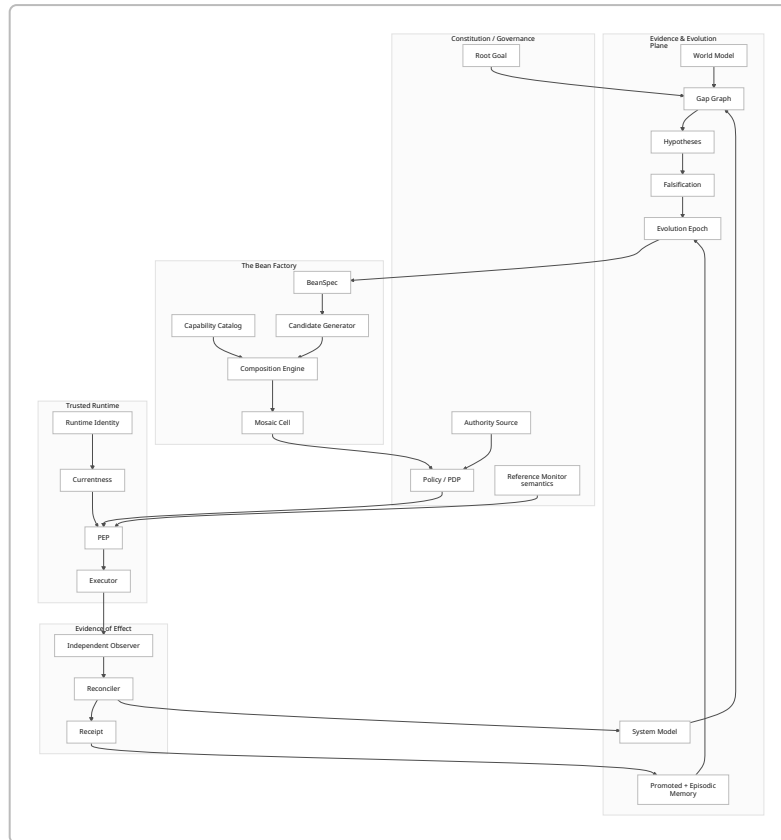
Nie sprowadza się do tego **pod warunkiem**, że trzy cechy pozostaną konstytutywne:

1. **tworzenie nowej capability wynika z falsyfikowalnego gapu, a nie tylko konfiguracji,**
2. **organizacja jest produktem problemu, nie predefiniowanym workflow,**
3. **ewolucja kodu i organizacji nie może ewoluować źródła własnego authority.**

Bez tych trzech własności nazwa „The Bean Factory” nie wnosi wystarczającej nowej semantyki.

Docelowa architektura, luka implementacyjna i plan plików

Target Architecture graph



Implementation Gap

Capability	Live master	Target Bean Factory	Gap
provenance-aware market evidence	zaimplementowane startup-specific	generic WorldEvidence	adapter/ generalizacja
generic evidence contract	zaimplementowane	reuse	mały
hypothesis/counter-evidence/falsifier	zaimplementowane	reuse	mały
experiment/result/falsification	zaimplementowane	reuse	mały
negative evidence memory	zaimplementowane	reuse	mały
epoch transitions	zaimplementowane	reuse	mały
EvolutionDelta	zaimplementowane, no effect authority	reuse	mały
governed change proposal	zaimplementowane	reuse	mały
detached builder admission	zaimplementowane	reuse	mały

Capability	Live master	Target Bean Factory	Gap
exact repository PEP	konkretna reference ścieżka	general effect provider framework	średni; system-wide complete mediation NOT_PROVEN
AgentSpec	zaimplementowane	Bean subtype	mały adapter
MissionSpec / SwarmSpec	zaimplementowane	Mosaic Cell foundation	średni
canonical BeanSpec	code search nie znalazł	wymagane	najmniejszy jawny gap
BeanCandidate	częściowe analogi candidate build	canonical model	średni
BeanInstance	agent/runtime analogi	heterogeniczny canonical lifecycle	średni
generic CompositionContract	domain-specific compositions istnieją	required	średni
generic CompositionEngine	częściowe planner/ fleet mechanisms	capability/ authority/ epistemic aware	duży
WORLD/SYSTEM/GOAL canonical triad	elementy istnieją rozproszone	wymagane	duży
machine-readable SystemModel(t)	nie wykazano canonical implementation	required	UNKNOWN / target
multi-criteria EvolutionPressure	startup ma single score + heurystyki	Pareto/hard gates	średni
seed→unknown organization generation	brak evidence końcowego eksperymentu	centralna teza	NOT_PROVEN
self-evolution of architecture	candidate pathway częściowo istnieje	closed-loop architecture evolution	NOT_PROVEN
self-authorization prevention	silnie obecne w current boundaries	hard invariant	silna baza
complete mediation wszystkich LION effects	dokumentacja jawnie nie deklaruje	target	NOT_PROVEN

Najważniejszy gap nie brzmi więc „LION nie potrafi ewolucji”. To byłoby niezgodne z live code.

Brzmi:

LION posiada epistemic evolution, organizational agents/swarms oraz governed self-implementation, ale nie posiada jeszcze jednej, domain-independent, canonical unit of evolvable capability łączącej te trzy płaszczyzny.

Tą brakującą jednostką jest `BeanSpec`.

Minimal Implementation Sequence

Faza	Zmiana	Dlaczego w tej kolejności
BF-A	canonical <code>BeanSpec</code> contract + digest + negative tests	bez niego wszystko inne jest metaforą
BF-B	<code>BeanCandidate</code> + exact binding <code>spec_digest</code> ↔ <code>implementation_digest</code>	rozdziela intent od implementacji
BF-C	startup adapter <code>SoftwareBuildSpec</code> → <code>BeanSpec/BuildRequest</code>	udowadnia kompatybilność bez przepisywania startup agent
BF-D	<code>CompositionContract</code>	formalizuje relacje
BF-E	deterministic <code>CompositionEngine</code> dla istniejących Beans	najpierw composition bez generation
BF-F	evolution adapter <code>EvolutionDelta</code> → <code>Bean need</code>	łączy R&D z Factory
BF-G	local candidate epoch loop	full loop bez consequential external effect
BF-H	dynamic Mosaic formation	organization generated from required capabilities
BF-I	existing builder-entry path as candidate materialization provider	reuse E004
BF-J	governed exact attach przez istniejący repository PEP	self-implementation
BF-K	effect observation/reconciliation → next epoch	zamknięcie pętli
BF-L	architecture/system-model promotion	dopiero po działającym loop

Pierwszy działający epoch nie musi mutować `master`:

Evidence
→ Hypothesis
→ Experiment
→ Falsification
→ BeanSpec
→ Candidate
→ Local Verification

- Reconciliation
- Memory
- Next Epoch

To jest dużo bezpieczniejszy proof generatywności niż natychmiastowy self-modifying repository loop.

Exact File-Level Plan

Plan celowo **nie tworzy od razu** `cyber_lion/bean_factory/`. Istniejący projekt jest contracts-first, a `CONTRACT_MAP.md` wprost preferuje adaptory i wspólne kontrakty przed przepisywaniem providerów.

Path	N/C	Responsibility	Input → output	State ownership
<code>cyber_lion/contracts/bean.py</code>	NEW	<code>BeanSpec</code> , lineage, lifecycle constants, digest	declarative fields → sealed spec	none
<code>cyber_lion/contracts/bean_candidate.py</code>	NEW	bind exact spec to candidate implementation	spec digest + impl digest + evidence → candidate	none
<code>cyber_lion/contracts/bean_composition.py</code>	NEW	CompositionContract + relation vocabulary	Bean refs + mission + constraints → sealed composition	none
<code>cyber_lion/enterprise/bean_composition.py</code>	NEW	deterministic CompositionEngine	specs/catalog/mission → admissible composition	in-memory derivation only initially
<code>cyber_lion/enterprise/evolutionary_factory.py</code>	NEW	thin composition root connecting R&D epoch to Bean layer	verified delta → Bean need/ candidate plan	references only
<code>cyber_lion/startup_agent/bean_adapter.py</code>	NEW	compatibility bridge from startup build to Bean	ProductHypothesis/ Experiment/ SoftwareBuildSpec → generic contracts	none
<code>cyber_lion/enterprise/models.py</code>	CHANGE MINIMAL	declare AgentSpec→Bean compatibility helper, not inheritance rewrite	AgentSpec → BeanSpec projection	existing model state unchanged
<code>cyber_lion/contracts/evolutionary_rnd.py</code>	CHANGE LATER	optional typed <code>target_capability_refs</code> in EvolutionDelta vNext	R&D evidence → candidate need	current immutable records
<code>cyber_lion/enterprise/evolutionary_epoch.py</code>	CHANGE LATER	project Bean candidate state into epoch	candidate evidence → next state	existing engine

Path	N/C	Responsibility	Input → output	State ownership
<code>cyber_lion/contracts/events.py</code> or vNext schema	CHANGE LATER	Bean lifecycle event types	Bean lifecycle → event	event stor external
<code>cyber_lion/tests/test_bean_contract.py</code>	NEW	canonical spec tests	invalid/valid specs	none
<code>cyber_lion/tests/test_bean_candidate.py</code>	NEW	candidate substitution/replay	specs/candidates	none
<code>cyber_lion/tests/test_bean_composition.py</code>	NEW	capability/interface/authority rules	graph cases	none
<code>cyber_lion/tests/test_evolutionary_factory.py</code>	NEW	first epoch integration	generic evidence → candidate	in-memor
<code>cyber_lion/tests/test_startup_bean_adapter.py</code>	NEW	prove startup compatibility	startup plans	none
<code>cyber_lion/tests/test_bean_redteam.py</code>	NEW LATER	architectural attacks	malicious contracts	none

W szczególności **nie zmieniałbym jeszcze** `repository_mutation_pep.py`. Ta warstwa już ma znacznie mocniejsze semantics niż proponowany początkowy Bean layer; modyfikacja jej przed udowodnieniem Bean contracts niepotrzebnie zwiększyłaby blast radius.

Eksperymenty, metryki, roadmapa i pierwsza misja

Metrics

Metric	Definicja pomiarowa
Time to First Useful Bean	<code>first_reconciled_useful_outcome_at</code> - <code>epoch_opened_at</code>
Time to Evidence	<code>first_valid_evidence_at</code> - <code>experiment_started_at</code>
Information Gain per Evolution Cycle	zmiana jawnego uncertainty set/posterior per cykl; raportować jako wektor domenowy, nie universal scalar
Bean Reuse Rate	użycia istniejących zweryfikowanych specs / wszystkie Bean selections
Bean Supersession Rate	jawne supersessions / promoted Bean specs / jednostkę czasu
Unnecessary Bean Generation Rate	candidates bez admission i bez useful negative information / generated candidates
Composition Failure Rate	failed valid composition attempts / attempts

Metric	Definicja pomiarowa
Organization Reconfiguration Cost	compute + latency + state transfer + verification + epistemic loss
Coalition Formation Time	composition request → sealed admissible Mosaic
Capability Coverage	wymagane capabilities pokryte przez admitted composition / wszystkie required
Dependency Depth	najdłuższa ścieżka dependency DAG
Authority Exposure	time-integral aktywnej consequential capability surface; raport per action class
Observability Coverage	consequential effects z wymaganym pre/post observation / consequential effects
Effect Attribution	reconciled effects z jednoznacznym execution/mission/authority lineage / reconciled effects
Independent Verification Ratio	independently verified consequential candidates / consequential candidates
Reconciliation Closure Rate	effects osiągające reconciled closure / executed effects
Evolution Regression Rate	promoted epochs łamiące wcześniejszy invariant / promoted epochs
Goal Drift Incidents	wykryte nieautoryzowane goal-digest changes
Unknown State Age	<code>now - first_entered_UNKNOWN</code> per unresolved state
Architecture Delta Frequency	promoted architecture deltas / epoch/time
Useful Negative Result Rate	negative results zmieniające przyszły wybór / wszystkie negative results
Seed-to-Capability Expansion Ratio	zweryfikowane użyteczne capabilities niewpisane explicite w Seed / capability count Seed

Ostatnia metryka jest najważniejsza, ale również najłatwiejsza do „oszukania”. Dlatego numerator musi spełniać:

```

capability not explicitly encoded in Seed
AND
not merely renamed template
AND
independently verified
AND
used in a successful or useful-negative experiment

```

Nie liczymy liczby wygenerowanych klas ani agentów.

Experiments

Experiment	Start	Co ma powstać	Success	Falsification
Kernel contract	Evidence + Goal	hypothesis→experiment→BeanSpec	deterministic replay daje ten sam spec digest	Factory wymaga hard-coded workflow
Local candidate	BeanSpec	candidate + verifier evidence	exact binding + local acceptance	candidate nie da się sfalsyfikować lub potrzebuje external authority
Capability composition	3-5 istniejących Beans	minimal composition	wszystkie required capabilities, no authority widening	composition wymaga ręcznego topology template
Dynamic Mosaic	unknown task + capability catalog	role-separated cell	topology wynika z task requirements	topology z góry zakodowana
Seed→organization falsification	minimal Seed + unseen problem	nowa capability + Mosaic + reconciled outcome	useful solution not explicitly templated	ręczne dopisanie template'u konieczne

Szczegółowy eksperyment Seed → Organization

Input

```

seed:
  evidence_contract: true
  hypothesis_falsification: true
  epoch_machine: true
  bean_contract: true
  capability_resolution: true
  composition_rules: true
  authority_boundary: true
  observer_reconciler: true

goal:
  supplied_at_runtime: true

```

```
problem:
  withheld_from_factory_design: true
  no_predefined_workflow: true
  no_predefined_mosaic_topology: true

providers:
  heterogeneous: true
  include_deterministic_and_model_based: true

effect_authority:
  local_candidate_only_initially
```

Procedura

1. Zamrozić Seed SHA/spec digest.
2. Wybrać zadanie, którego workflow nie znajduje się w Seed/test fixtures.
3. Dostarczyć wyłącznie Goal + current observations + capability descriptors.
4. Zmierzyć `Gap`.
5. Wygenerować minimum dwie konkurencyjne hipotezy i counterhypothesis.
6. Wymusić jawne falsifiers.
7. Wybrać eksperyment według hard constraints + Pareto front.
8. Ustalić wymagane capabilities.
9. Sprawdzić catalog.
10. Dla brakującej capability wygenerować `BeanSpec`.
11. Zweryfikować, czy `BeanSpec` nie zawiera effect credential/authority.
12. Utworzyć candidate.
13. Zbudować Mosaic z minimum explorer/builder/verifier/observer potrzebnych dla task.
14. Uruchomić tylko local/bounded candidate experiment.
15. Zebrać independent result.
16. Falsyfikować hipotezę.
17. Reconcile expected vs observed.
18. Zapisać positive i negative evidence.
19. Otworzyć następną epokę.
20. Sprawdzić, czy organizacja zmieniła się zgodnie z nowym gapem.

Success criteria

- S1: Problem workflow not represented as a Seed template.
- S2: At least one required capability is derived after observing the problem.
- S3: Mosaic topology differs from at least one previous experiment.
- S4: BeanSpec is reconstructable from evidence/hypothesis/experiment lineage.
- S5: Candidate cannot mint authority.
- S6: Builder is not sole verifier.
- S7: Negative evidence survives next epoch.
- S8: Same immutable inputs replay to same deterministic structural decisions where no probabilistic provider is involved.
- S9: Observed outcome changes next epoch decision.
- S10: No external effect is required to prove initial generativity.

Failure criteria

F1: New problem requires adding a hard-coded workflow to Factory.
F2: Required Bean type had to be predeclared as a concrete template.
F3: Organization topology is effectively constant.
F4: Capability generation is only renaming/copying a prior Bean.
F5: Factory bypasses independent verifier to progress.
F6: Failure causes success metric to be rewritten.
F7: Missing observer causes wider authority.
F8: Negative evidence disappears after summarization.
F9: Same Bean identity silently changes implementation.
F10: Generated candidate reaches consequential effect without canonical authority.

To jest centralna falsyfikacja całej idei:

Jeżeli nowy typ problemu zawsze wymaga dopisania nowego template'u Factory, to nie powstał system generatywny — powstał rozbudowany scaffold generator.

Evolution Roadmap

```
CURRENT MASTER
|
├─ mature epistemic R&D contracts
├─ startup minimum-experiment/minimum-build pattern
├─ agent/mission/swarm organizational contracts
├─ controlled builder entry
├─ exact repository PEP reference path
|
▼
CANONICAL BEAN CONTRACT
|
▼
BEAN CANDIDATE + LINEAGE
|
▼
GENERIC COMPOSITION CONTRACT
|
▼
STATIC CAPABILITY COMPOSITION
|
▼
R&D → BEAN NEED ADAPTER
|
▼
LOCAL CLOSED EPOCH
|
▼
DYNAMIC MOSAIC FORMATION
|
```

```

▼
GOVERNED SELF-IMPLEMENTATION
|
▼
POST-EFFECT RECONCILIATION
|
▼
SYSTEM MODEL RECONSTRUCTION
|
▼
ARCHITECTURE DELTA PROMOTION
|
▼
SELF-OBSERVING EVOLUTIONARY LION

```

Dopiero końcowe stadium uzasadniałoby mocne zdanie „LION sam rozwija swoją architekturę”. Na aktualnym `master` właściwszy status to:

governed self-implementation pathway: częściowo i mocno zaimplementowany;
autonomous architecture evolution closed loop: NOT_PROVEN.

To jest także zgodne z ostrożnością samej dokumentacji: green CI, tested sandbox contract i workload identity nie dowodzą complete mediation, OS isolation ani production attestation wszystkich ścieżek.

First Governed Mission

Najmniejszy zweryfikowany gap na HEAD `a94882bb...` to **brak canonical, domain-independent BeanSpec**, przy jednoczesnym istnieniu jego wyspecjalizowanych prekursorów (`AgentSpec`, `SoftwareBuildSpec`, capability descriptors, `EvolutionDelta`). Code search nie znalazł canonical `BeanSpec`; dlatego jest to właściwy pierwszy krok, a nie tworzenie całego `bean_factory/`. Obecny `AgentSpec` pokazuje dokładnie, jak wiele pól można reuse, ale jego semantyka jest agent-specific.

```

mission_id: BF-M001-CANONICAL-BEAN-SPEC

classification:
  change_class: SELF_UPGRADE_FOUNDATION
  risk: GREEN
  authority_effect: NONE
  execution_effect: NONE
  repository_ref_effect: NONE
  external_effect: NONE

baseline:
  repository: DonkeyJJLove/ai_platform
  branch: master
  head_sha: a94882bb80349482c287b76e027c93fe1ed6f1fe
  tree_sha: 886fbefc79b82b405e0400e53fe95bfc420f4c67

verified_gap:

```

```

statement: >
    LION has domain-specific AgentSpec and SoftwareBuildSpec plus generic
    evolutionary contracts, but no canonical domain-independent BeanSpec
    was found that can represent agents, deterministic components,
    observers, verifiers, adapters and tools under one non-effectful
    capability contract.
status: OBSERVED_GAP
evidence:
    - cyber_lion/enterprise/models.py
    - cyber_lion/startup_agent/build_planner.py
    - cyber_lion/contracts/evolutionary_rnd.py
    - cyber_lion/CAPABILITY_MAP.md

candidate_scope:
    - cyber_lion/contracts/bean.py
    - cyber_lion/tests/test_bean_contract.py

explicit_non_goals:
    - no Bean runtime
    - no CompositionEngine
    - no repository mutation
    - no deployment
    - no new PDP
    - no PEP modification
    - no AgentSpec migration
    - no startup_agent rewrite
    - no credentials
    - no dynamic code execution
    - no self-registration
    - no master/ref mutation by candidate builder

deliverable:
    BeanSpec:
        required_properties:
            - frozen/immutable dataclass
            - canonical domain-separated digest
            - bean_id
            - bean_type
            - version
            - lineage
            - purpose
            - goal_contract
            - success_condition
            - stop_condition
            - defer_condition
            - inputs
            - outputs
            - interfaces
            - required_capabilities
            - provided_capabilities
            - authority_ceiling

```

- required_grants
- epistemic_requirements
- evidence_requirements
- provenance_policy
- memory_policy
- context_policy
- observability_requirements
- resource_budget
- cost_budget
- time_budget
- runtime_class
- sandbox_class
- dependencies
- compatibility_constraints
- failure_modes
- degradation_policy
- revocation_policy
- security_invariants
- acceptance_tests
- falsification_conditions
- evolution_hooks
- replacement_policy
- supersession_policy
- implementation_digest
- spec_digest

security_invariants:

- BeanSpec is descriptive and cannot grant authority
- authority_ceiling is a ceiling, never a credential
- required_grants are references/classes, never secret material
- spec cannot contain execution-effect material
- lineage cannot silently substitute parent digest
- duplicate set-like fields are rejected
- consequential authority requires explicit observability requirements
- spec_digest covers every semantic field
- changing any semantic field changes spec_digest

negative_tests:

- empty bean_id denied
- duplicate capabilities denied
- malformed digest denied
- digest substitution denied
- credential-like material denied where prohibited
- unsafe authority class denied
- authority ceiling without observability denied for consequential class
- duplicate lineage parent denied
- self-supersession denied
- invalid lifecycle/replacement policy denied

acceptance:

- python -m compileall -q cyber_lion

- `python -m unittest cyber_lion.tests.test_bean_contract -v`
- `python -m unittest discover -s cyber_lion/tests -p "test_*.py" -v`
- existing Startup Evolution demo remains green
- no existing contract semantics changed
- no public execution method introduced
- deterministic spec digest demonstrated
- mutation of each semantic field alters digest

falsification:

- >
Reject this mission design if an existing canonical live contract can represent the same heterogeneous capability unit without agent-specific semantics and without weakening authority boundaries.
- >
Reject BeanSpec abstraction if removing it causes no semantic loss relative to existing AgentSpec/CapabilityDescriptor contracts.

promotion:

- candidate_build: allowed only as detached/local candidate
- merge: requires separate repository authority
- release: forbidden
- deploy: forbidden

Ta misja jest celowo mała. Nie próbuje jeszcze udowodnić całej Bean Factory. Jej wartość polega na tym, że tworzy **pierwszy brakujący obiekt ontologiczny**, bez którego następne kroki — `BeanCandidate`, `CompositionContract`, generowanie Mosaic i `seed` → organization experiment — nie mają stabilnej granicy kontraktowej.

Ostateczna ocena wzorca po tym researchu brzmi:

The Bean Factory = Evolutionary Capability Factory + Dynamic Organization Grammar + Governed Self

a nie:

Bean Factory = Agent Factory

ani:

Bean Factory = Self-Modifying LLM

ani:

Bean Factory = Autonomous CI/CD.

Najgłębsza własność, której LION powinien próbować dowieść eksperymentalnie, jest następująca:

Z bardzo małego, stabilnego konstytucyjnego Seed system potrafi wyhodować złożoność funkcjonalną i organizacyjną, której Seed nie kodował explicite; jednocześnie żadna nowo wygenerowana złożoność nie daje sobie sama większej władzy.

Obecny `master` dostarcza już zaskakująco dużej części drugiej połowy tego twierdzenia — separacji epistemiki od authority, falsyfikacji, pamięci, builder-entry i exact-effect enforcement.

Nieudowodniona pozostaje pierwsza połowa: że z minimalnego Seed rzeczywiście powstanie nowa capability i nowa organizacja, zamiast jedynie odtworzenia wcześniej zakodowanych template'ów.

Dlatego właściwym naukowym milestone'em dla The Bean Factory nie jest „system sam zmodyfikował kod”.

Jest nim:

UNSEEN PROBLEM → NEW VERIFIED CAPABILITY → NEW MOSAIC → RECONCILED RESULT

bez zmiany Seed i bez samonadania authority.

Dopiero pozytywny wynik takiego eksperymentu uzasadni stwierdzenie, że LION nie tylko automatyzuje rozwój software'u, lecz rzeczywiście posiada **gramatykę wzrostu organizacji**.

1 3 <https://docs.spring.io/spring-framework/docs/7.0.x/javadoc-api/org/springframework/beans/factory/BeanFactory.html>

<https://docs.spring.io/spring-framework/docs/7.0.x/javadoc-api/org/springframework/beans/factory/BeanFactory.html>

2 5 <https://research.ibm.com/publications/an-architectural-approach-to-autonomic-computing>

<https://research.ibm.com/publications/an-architectural-approach-to-autonomic-computing>

4 <https://docs.osgi.org/specification/osgi.core/8.0.0/framework.service.html>

<https://docs.osgi.org/specification/osgi.core/8.0.0/framework.service.html>

6 <https://research.ibm.com/publications/a-multi-agent-systems-approach-to-autonomic-computing>

<https://research.ibm.com/publications/a-multi-agent-systems-approach-to-autonomic-computing>

7 <https://i.stanford.edu/TR/cstr8x.html>

<https://i.stanford.edu/TR/cstr8x.html>

8 <https://www.sei.cmu.edu/library/architectural-evaluation-of-collaborative-agent-based-systems/>

<https://www.sei.cmu.edu/library/architectural-evaluation-of-collaborative-agent-based-systems/>

9 <https://www.sei.cmu.edu/library/consistency-in-dynamic-reconfiguration/>

<https://www.sei.cmu.edu/library/consistency-in-dynamic-reconfiguration/>